UNIVERSIDAD LA SALLE

FACULTAD DE INGENIERÍA Y ARQUITECTURA
CARRERA PROFESIONAL DE INGENIERÍA DE SOFTWARE

EXAMEN PARCIAL

Apellidos y nombres: ROGER INFA SANCHEZ

Curso: Web Semánticas y Ontologías
Semestre: Octavo
Docente: Josue Miguel Flores Parra
Fecha: 22-10-2026

1 Problema 1. Lógica de predicados

1.1 a) Representación lógica

Vocabulario utilizado:

- `Veterinario(x)`: x es veterinario.
- `Animal(x)`: x es animal.
- `AnimalRescatado(x)`: x es animal rescatado.
- `Atiende(x, y)`: x atiende a y.
- `Vacunado(x)`: x ha recibido una vacuna.

1. Todos los veterinarios atienden al menos a un animal.

```
∀x (Veterinario(x) → ∃y (Animal(y) ∧ Atiende(x, y)))
```

Justificación:

- ∀x porque es una afirmación universal sobre todos los veterinarios.
- Se usa implicación → y no conjunción, porque solo se exige algo a quienes cumplen `Veterinario(x)`.
- ∃y porque basta con que exista al menos un animal atendido, no todos.
- `Animal(y) ∧ Atiende(x, y)` garantiza que ese individuo sea animal y que exista la relación de atención.

2. Existe al menos un animal rescatado que ha recibido una vacuna.

```
∃x (AnimalRescatado(x) ∧ Vacunado(x))
```

Justificación:

- ∃x porque basta un caso para que la afirmación sea verdadera.
- Se usa conjunción ∧ porque el mismo individuo debe cumplir ambas propiedades a la vez.
- No se usa implicación, porque no es una regla general sino la afirmación de existencia de un individuo con dos características.

1.2 b) Identificación y corrección de errores

Afirmación a representar: *"Todos los animales que reciben una vacuna son animales rescatados"*.

Expresión propuesta:

```
∀x (Vacunado(x) ∧ AnimalRescatado(x))
```

Por qué es incorrecta:

1. Usa conjunción ∧ con cuantificador universal. Eso afirma que **todo individuo del universo** cumple ambas cosas a la vez: todo es vacunado y todo es animal rescatado.
2. No expresa condicionalidad. La frase original es un "si … entonces …": *si* está vacunado, *entonces* es rescatado. La fórmula propuesta no tiene → y por tanto no restringe, sino que generaliza en exceso.
3. Bastaría un contraejemplo (una persona no vacunada, un animal no rescatado) para hacer falsa la fórmula, aunque la afirmación original podría seguir siendo verdadera.

Expresión corregida:

$\forall x\ (\text{Vacunado}(x) \rightarrow \text{AnimalRescatado}(x))$

Variante más explícita delimitando a animales:

$\forall x\ ((\text{Animal}(x) \wedge \text{Vacunado}(x)) \rightarrow \text{AnimalRescatado}(x))$

Justificación de la corrección: con → solo se exige `AnimalRescatado(x)` a quienes cumplen el antecedente. Los individuos no vacunados no falsan la fórmula, que es exactamente lo que pide el enunciado.

---

2 Problema 2. Diseño de una ontología — Repositorio universitario

Dominio: organizar publicaciones para clasificar y consultar; distinguir digitales e impresas, registrar autores, identificar formatos, reconocer autoría múltiple y licencia abierta.

2.1 a) Clases necesarias y propósito

Clase	Propósito
`Publicacion`	Clase central. Toda obra del repositorio.
`PublicacionDigital`	Publicaciones en soporte digital. Permite clasificar y aplicar reglas propias de lo digital.
`PublicacionImpresa`	Publicaciones en soporte físico. Incompatible con la anterior.
`PublicacionMultiautor`	Clase definida para las publicaciones con ≥2 autores distintos. Se infiere automáticamente.
`PublicacionDigitalPDF`	Clase definida para publicaciones digitales disponibles en PDF. Se infiere automáticamente.
`Autor`	Personas que escriben publicaciones. Se vincula vía `escritoPor`.
`Formato`	Formatos de disponibilidad. Ej.: `PDF`, `EPUB`, `HTML`, `TapaBlanda`, `TapaDura`.
`Licencia`	Licencias de uso de la publicación.
`LicenciaAbierta`	Subclase de `Licencia` para acceso abierto (ej. Creative Commons). Permite filtrar qué se puede difundir libremente.

Individuos ejemplo (a usar en Protégé, Problema 4): `pub1`, `pub2`, `autor1`, `autor2`, `formatoPDF`.

2.2 b) Relaciones entre conceptos

Propiedad (dominio → rango)	Qué información representa
`escritoPor` (`Publicacion → Autor`)	Autoría. Quién escribió cada publicación. Base para `PublicacionMultiautor`. Inversa: `esAutorDe`.
`disponibleEn` (`Publicacion → Formato`)	En qué formato(s) está disponible cada publicación. Base para `PublicacionDigitalPDF`.
`tieneLicencia` (`Publicacion → Licencia`)	Bajo qué licencia se publica. Permite identificar licencia abierta.

Se modelan como `ObjectProperty`, no como cadenas de texto, para poder razonar sobre los individuos relacionados y mantener integridad referencial.

2.3 c) Jerarquía de clases y restricciones

Representación textual:

```
Publicacion
  ├─ PublicacionDigital
  ├─ PublicacionImpresa
  ├─ PublicacionMultiautor      [definida]
  └─ PublicacionDigitalPDF      [definida]

Autor
Formato
  ├─ (individuos: PDF, EPUB, HTML, TapaBlanda, ...)
Licencia
  └─ LicenciaAbierta
```

Restricciones (Manchester):

```
1. PublicacionDigital SubClassOf Publicacion
2. PublicacionImpresa SubClassOf Publicacion
3. PublicacionDigital DisjointWith PublicacionImpresa
4. PublicacionMultiautor EquivalentTo Publicacion and (escritoPor min 2 Autor)
5. PublicacionDigitalPDF EquivalentTo PublicacionDigital and (disponibleEn value P
   // alternativa con clase: ... and (disponibleEn some FormatoPDF)
6. Publicacion SubClassOf (escritoPor some Autor)
7. Publicacion SubClassOf (disponibleEn some Formato)
8. LicenciaAbierta SubClassOf Licencia
9. PublicacionConLicenciaAbierta EquivalentTo Publicacion and (tieneLicencia some
```

Coherencia que aportan:

- 1-3 garantizan la distinción digital/impreso y la incompatibilidad exigida en el Problema 4.
- 4-5 son clases definidas (`EquivalentTo`), permiten clasificación automática por HermiT.
- 6-7 evitan publicaciones sin autor ni formato.
- 9 permite consultar qué se puede difundir en abierto.

2.4 d) Justificación de dos decisiones de diseño

Decisión 1: `Formato` como clase con individuos (`PDF`, …) y no como `xsd:string`. Si `formato` fuera un dato literal (`titulo`, `"PDF"`), no se podría usar `disponibleEn value PDF` ni `some FormatoPDF` para definir `PublicacionDigitalPDF`, ni distinguir formatos digitales de impresos con jerarquía. Con clase, el razonador puede inferir y se puede extender (nuevo formato = nuevo individuo, sin cambiar el esquema).

Decisión 2: `PublicacionMultiautor` y `PublicacionDigitalPDF` como clases definidas (`EquivalentTo`) y no como subclases primitivas (`SubClassOf`). Si fueran primitivas, habría que clasificar manualmente cada individuo y se podría olvidar o equivocar. Como definidas (condición necesaria y suficiente), HermiT clasifica solo cuando se cumplen `min 2` autores o `Digital + PDF`. Sin `EquivalentTo` no habría inferencia automática, que es justo lo que piden los requisitos funcionales del Problema 4.

---

3 Problema 3. Interpretación de axiomas OWL

3.1 a) Subclases

```
PublicacionDigital SubClassOf Publicacion
```

Significa que todo individuo de `PublicacionDigital` es también individuo de `Publicacion`. Es inclusión de conjuntos, no igualdad: puede haber publicaciones que no sean digitales (las impresas). La subclase hereda las restricciones de la superclase: si `Publicacion` exige `escritoPor some Autor`, toda digital también debe cumplirlo.

3.2 b) Restricciones `some` vs `only`

- `escritoPor some Autor` (existencial / `someValuesFrom`): Garantiza que **existe al menos un** valor de `escritoPor` que es `Autor`. No garantiza que *todos* lo sean, ni cuántos hay. Una publicación con un autor válido y otros valores no-autor seguiría cumpliéndola.
- `escritoPor only Autor` (universal / `allValuesFrom`): Garantiza que **todos** los valores de `escritoPor`, si existen, son de tipo `Autor`. No garantiza existencia: si la publicación no tiene ningún `escritoPor`, la restricción se cumple por vacío (verdad vacuosa).

En la práctica se combinan: `escritoPor some Autor` para exigir autor, `escritoPor only Autor` para prohibir valores no-autor.

3.3 c) Cardinalidad

1. Definición OWL adecuada (Manchester):

```
Class: PublicacionMultiautor
  EquivalentTo: Publicacion and (escritoPor min 2 Autor)
```

Se usa `EquivalentTo` (no solo `SubClassOf`) para que el razonador infiera automáticamente la pertenencia cuando se cumple el mínimo. `min 2` exige al menos dos valores distintos en `Autor`.

2. Por qué dos nombres no bastan:

OWL **no asume nombre único (UNA)**. Dos IRI distintos (`autor1`, `autor2`) pueden denotar al mismo individuo salvo que se declare lo contrario. El razonador puede fusionarlos y contar 1 solo autor.

Para que cuenten como 2 se debe declarar:

```
DifferentIndividuals: autor1, autor2
```

o bien:

```
autor1 DifferentFrom autor2
```

Sin esto, `escritoPor min 2` podría no dispararse aunque se hayan registrado dos nombres.

3.4 d) Equivalencia de clases

`EquivalentTo` define condición **necesaria y suficiente**. No solo dice "todo miembro cumple esto" (como `SubClassOf`), sino también "todo lo que cumple esto es miembro". Eso permite al razonador **clasificar automáticamente** individuos que no fueron tipados explícitamente.

Ejemplo del repositorio:

```
Class: PublicacionDigitalPDF
  EquivalentTo: PublicacionDigital and (disponibleEn value PDF)
```

Si se registra `pub1 type PublicacionDigital` + `pub1 disponibleEn PDF`, sin decir que es `PublicacionDigitalPDF`, HermiT lo infiere como miembro. Con solo `SubClassOf` esa inferencia no ocurriría.

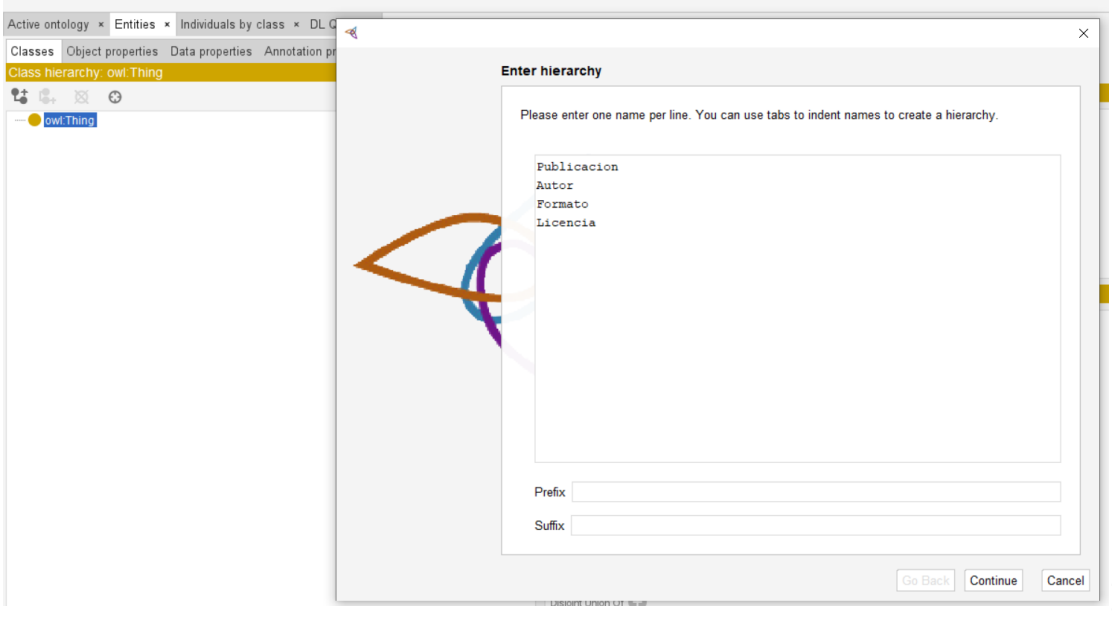
---

4 Problema 4. Construcción y razonamiento en Protégé

Ontología construida desde cero en Protégé. IRI: `http://www.semanticweb.org/user/ontologies`
Archivo de entrega: `Roger.owl` (mismo modelo del Problema 2).

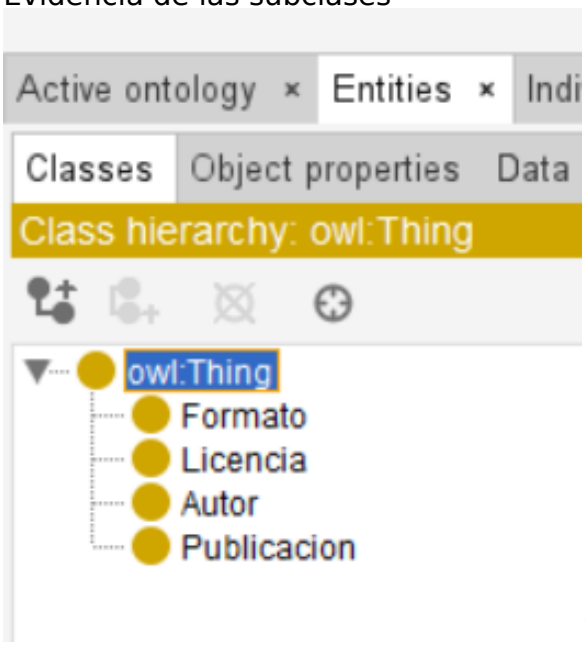
4.1 a) Modelo construido e individuos registrados

Creación de las 4 subclases del caso en pestaña Entities/clasess



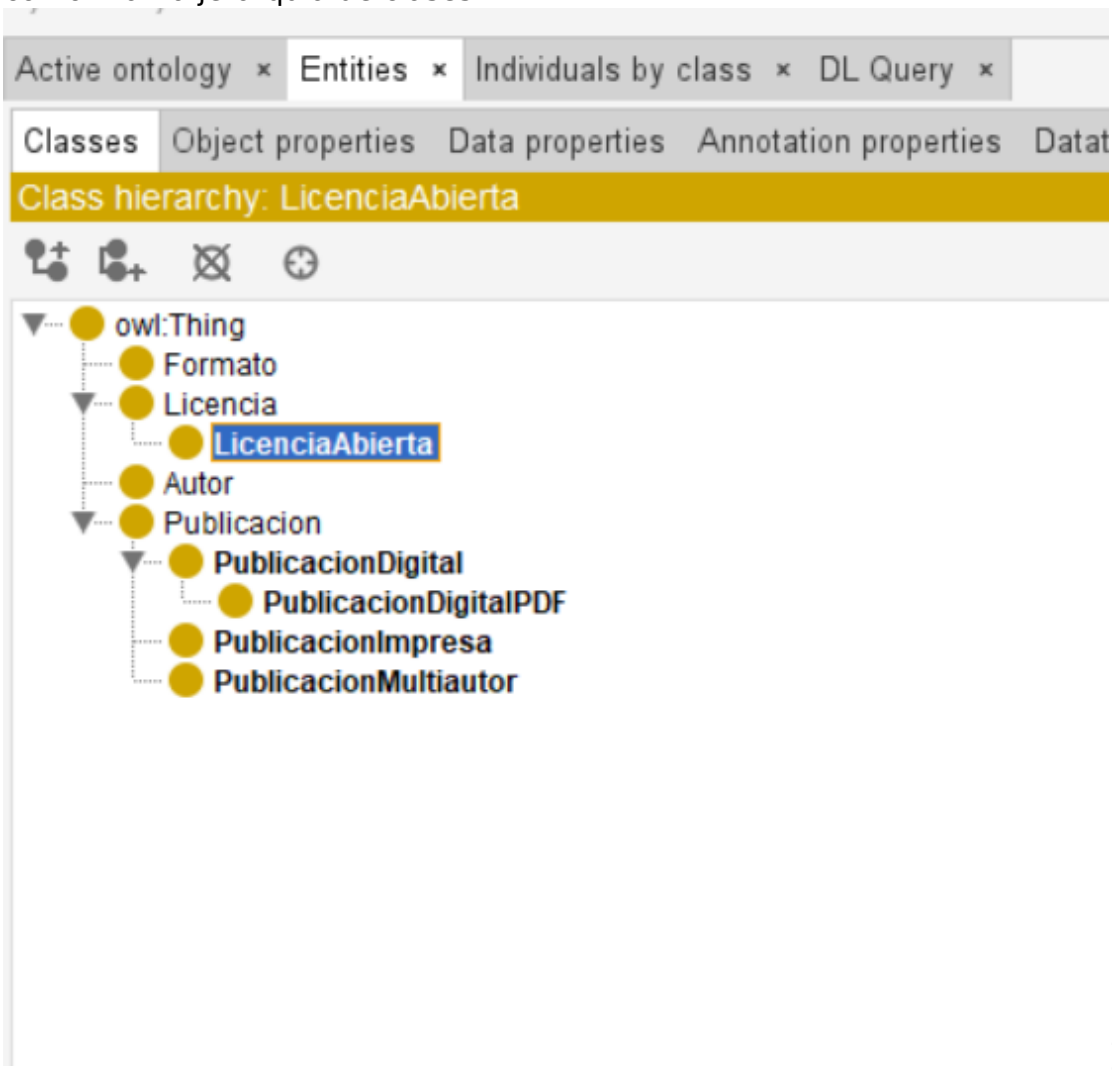
}

Evidencia de las subclases



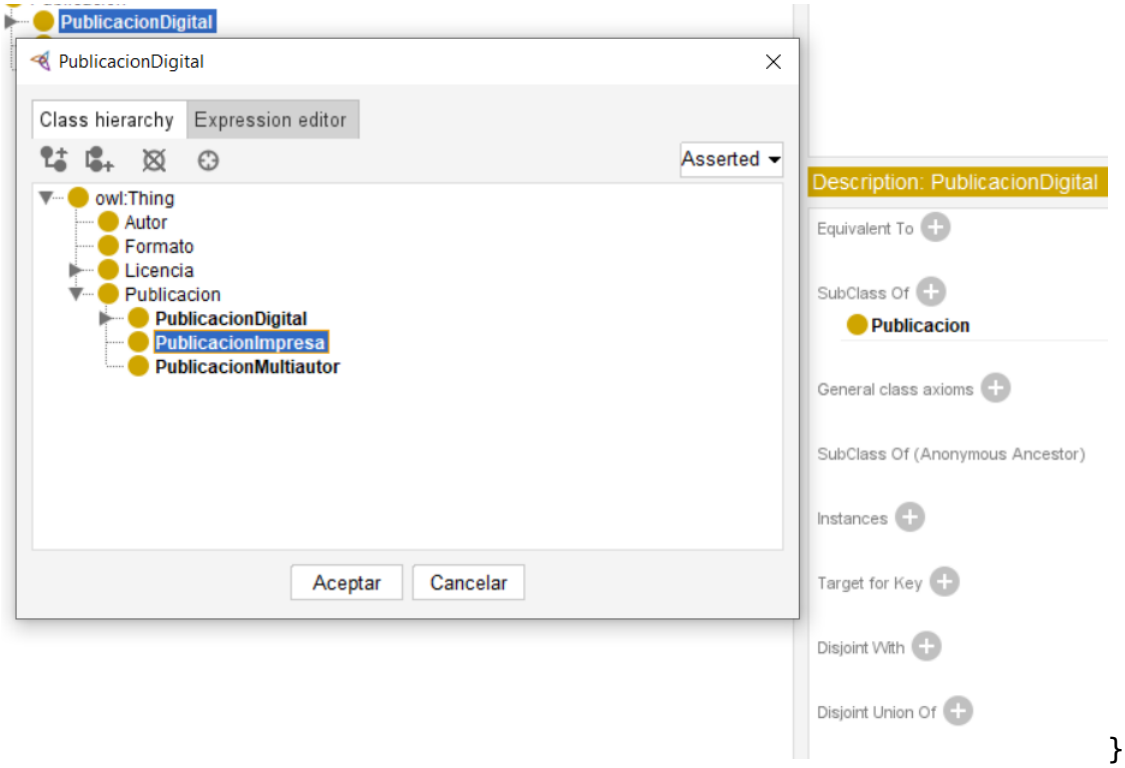
}

Creación de las subclases de Publicacion, Licencia y publicaciónDigital para terminar de conformar la jerarquía de clases



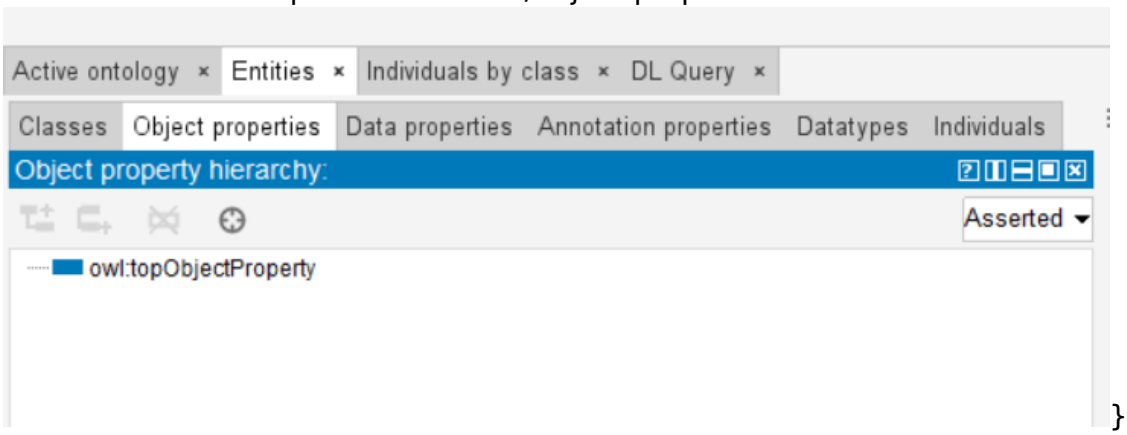
}

Configuración de la disjunción en PublicacionDigital



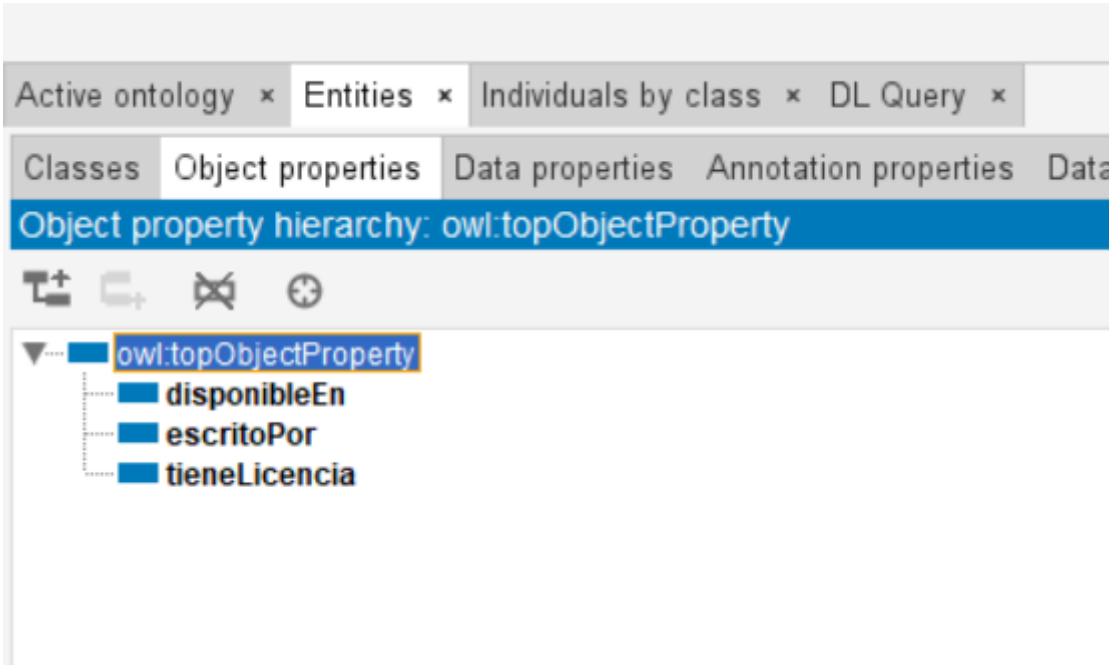
}

Creación de las propiedades en pestañas object properties y data properties:
Nos situamos en la pestaña Entities/object properties



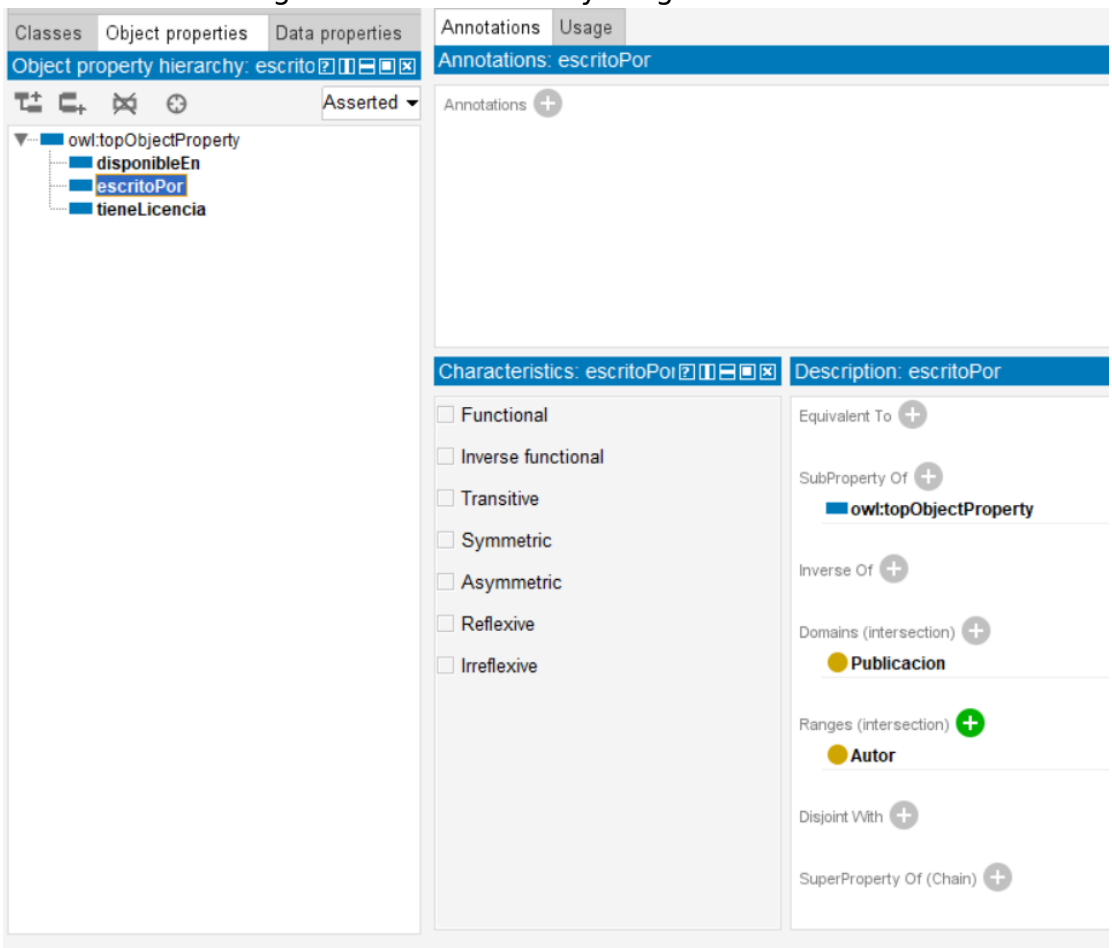
}

Y creamos 3 propiedades donde posteriormente pasaremos a configurar el dominio y rango

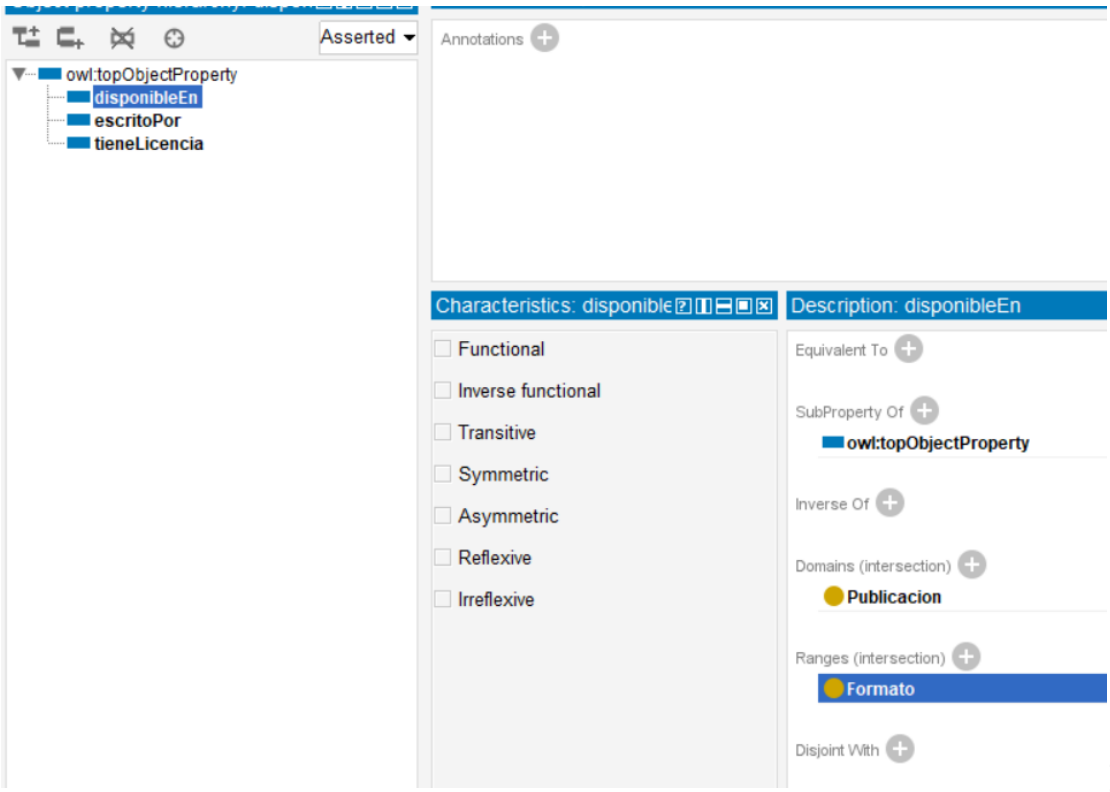


}

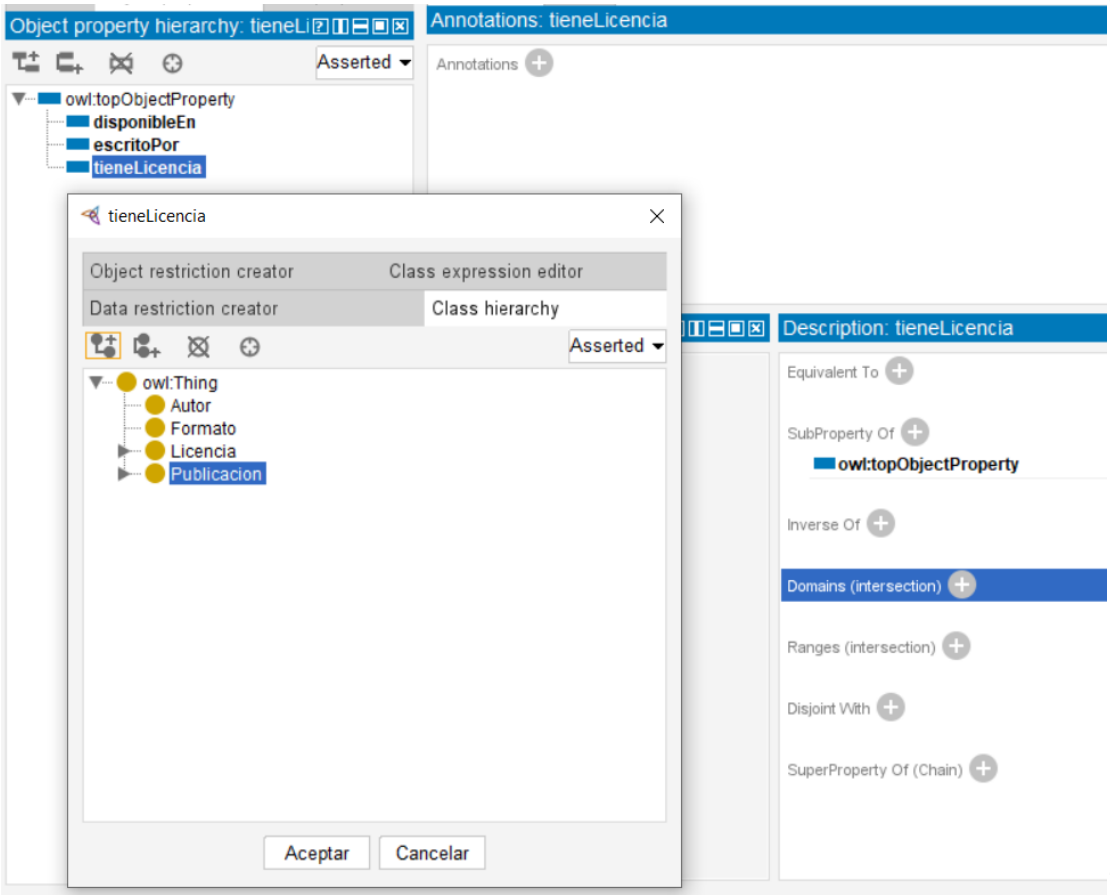
Mostrando la configuración de dominio y rango



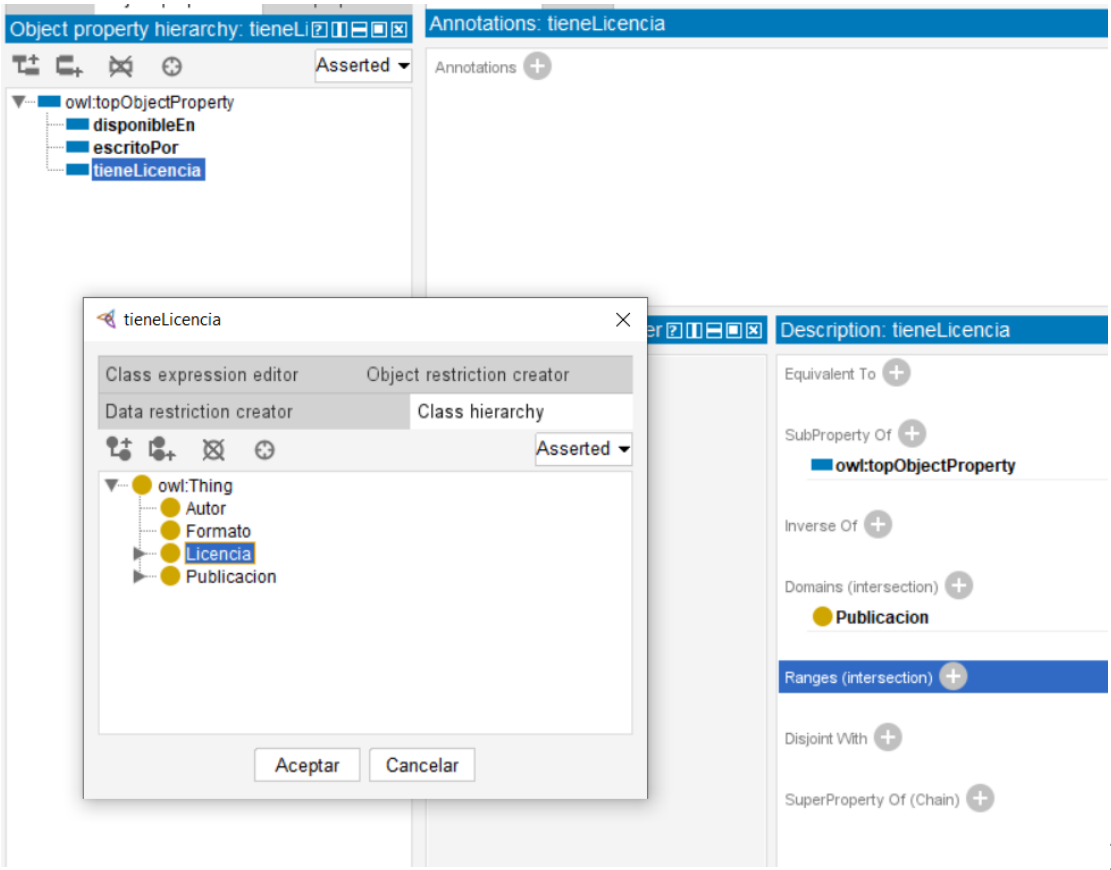
}


Asserted
owl:topObjectProperty
disponibleEn
escritoPor
tieneLicencia
Annotations
Characteristics: disponible
Functional
Inverse functional
Transitive
Symmetric
Asymmetric
Reflexive
Irreflexive
Description: disponibleEn
Equivalent To
SubProperty Of
owl:topObjectProperty
Inverse Of
Domains (intersection)
Publicacion
Ranges (intersection)
Formato
Disjoint With

}

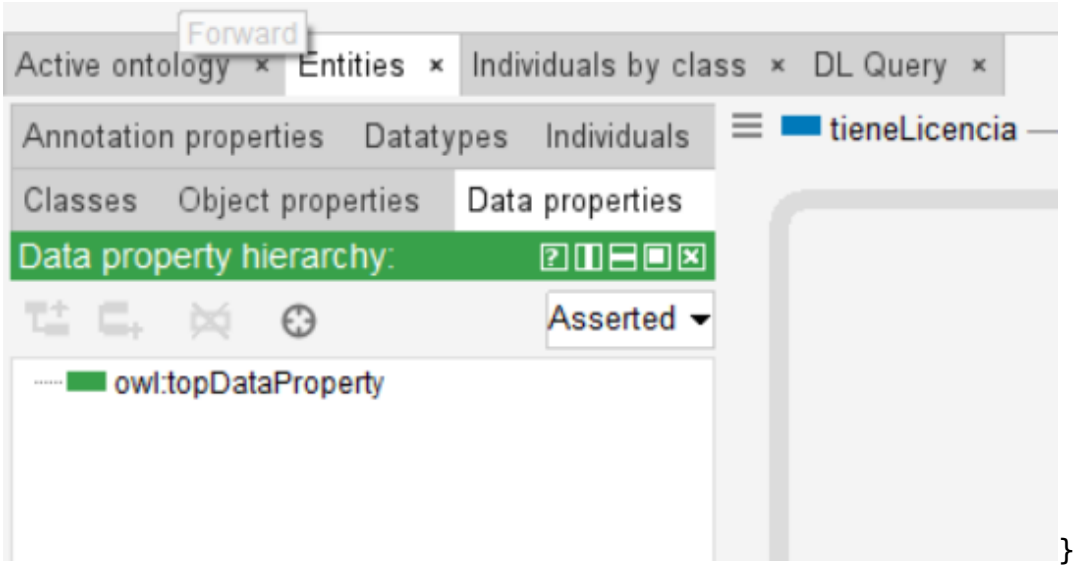

Object property hierarchy: tieneLi
Asserted
owl:topObjectProperty
disponibleEn
escritoPor
tieneLicencia
Annotations: tieneLicencia
Annotations
tieneLicencia
Object restriction creator
Class expression editor
Data restriction creator
Class hierarchy
Asserted
owl:Thing
Autor
Formato
Licencia
Publicacion
Aceptar
Cancelar
Description: tieneLicencia
Equivalent To
SubProperty Of
owl:topObjectProperty
Inverse Of
Domains (intersection)
Ranges (intersection)
Disjoint With
SuperProperty Of (Chain)

}



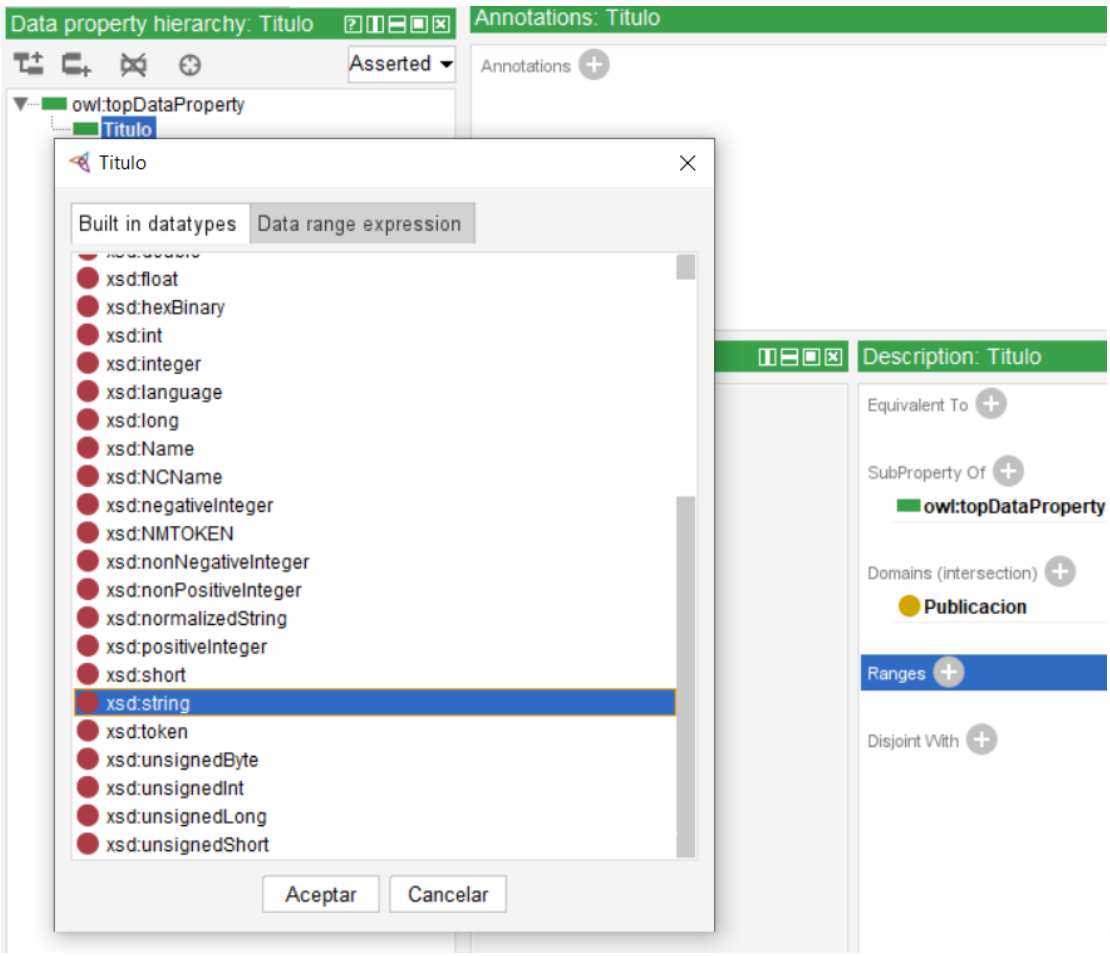
}

Ahora en Data properties



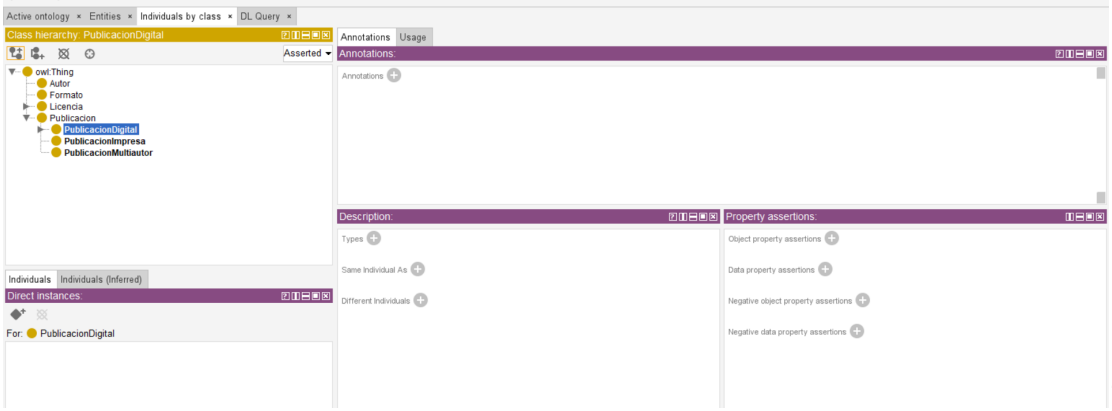
}

Creamos la propiedad titulo configurando su dominio y rango



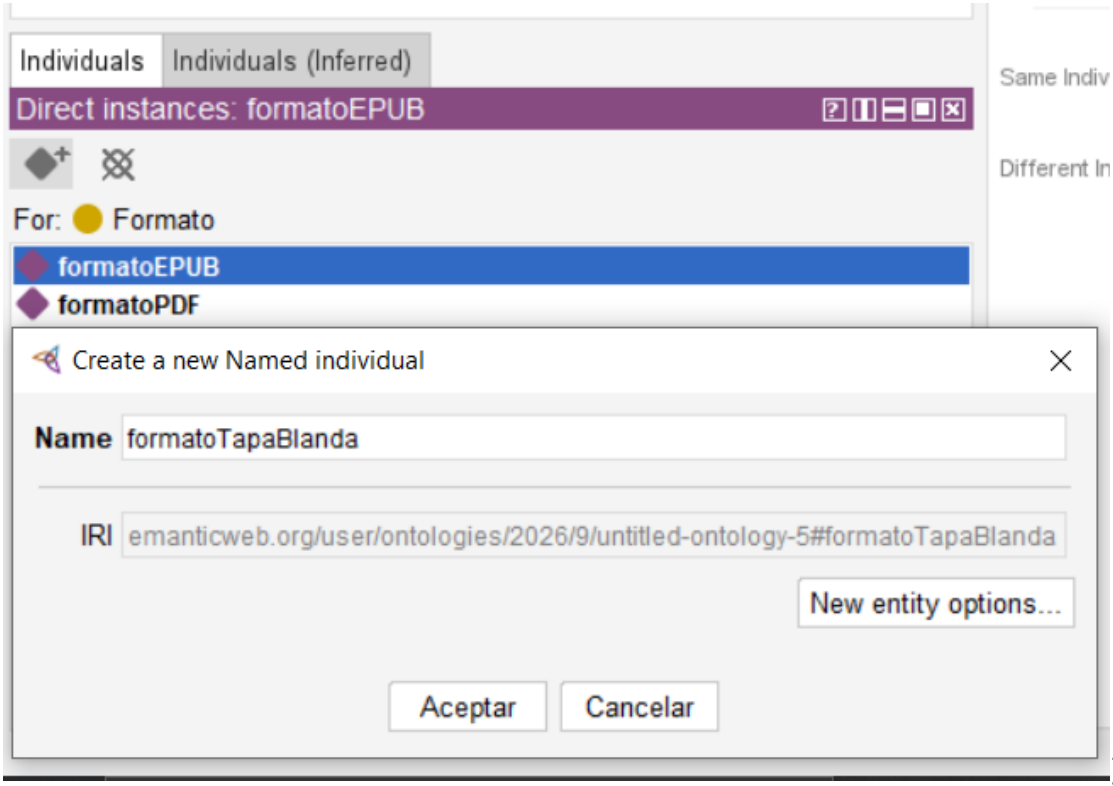
}

Seguidamente iremos a la pestaña Individuals by class para crear los individuos de apoyo y diferenciarlos

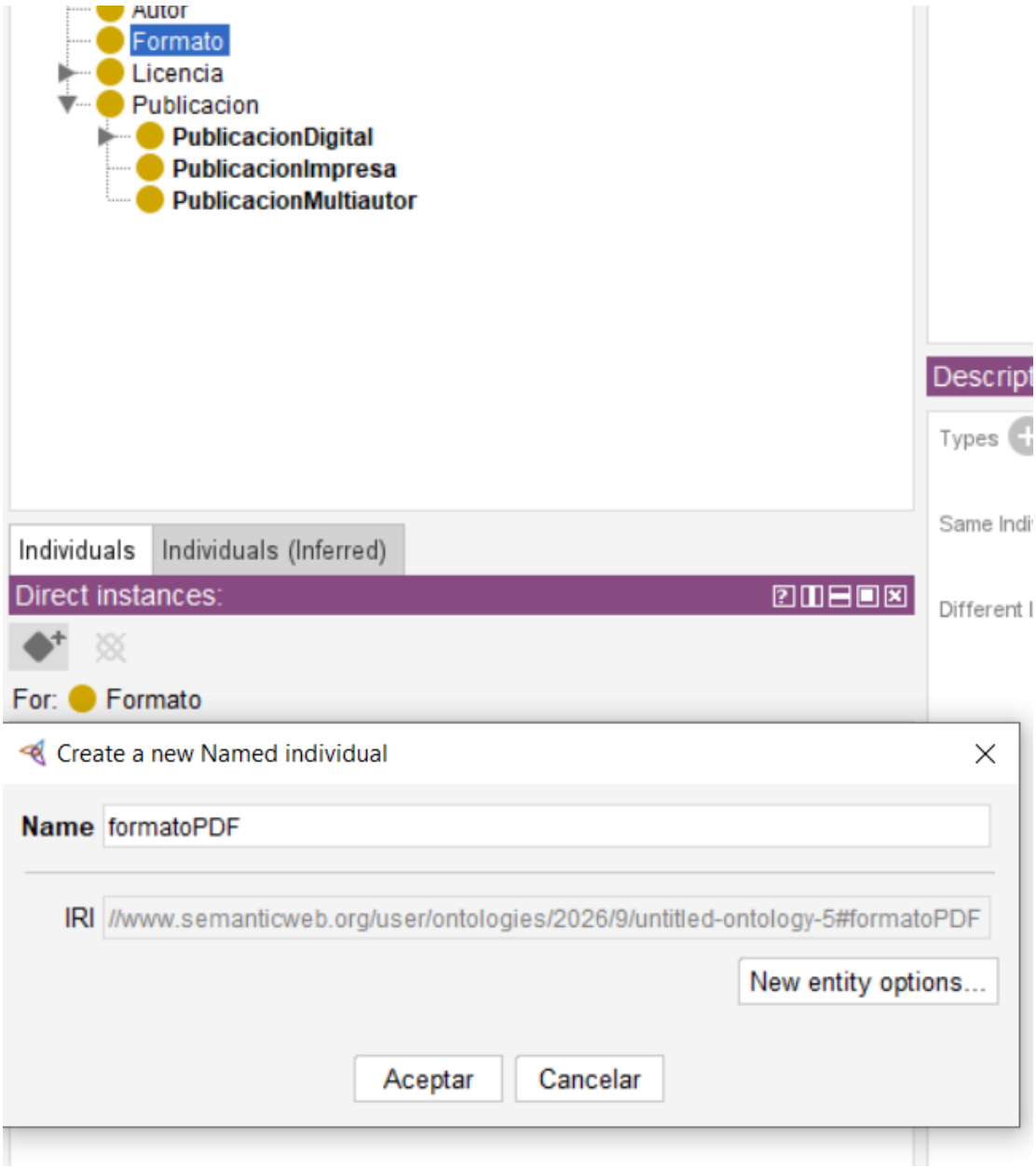


}

Iniciando con la creación de 3 individuos para la subclase Formato, la creación es presionando el rombo gris con un + arriba a la derecha

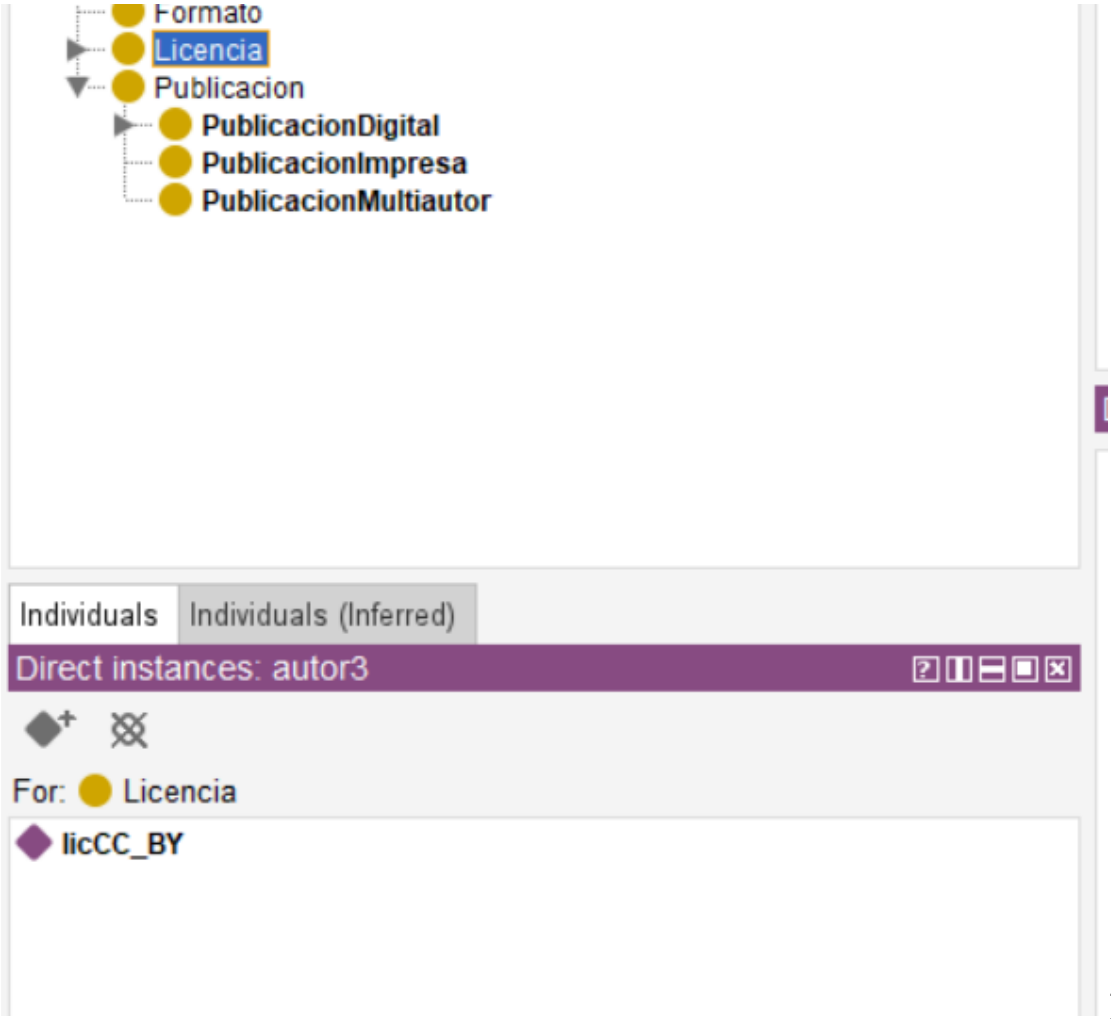

Individuals
Individuals (Inferred)
Direct instances: formatoEPUB
For: Formato
formatoEPUB
formatoPDF
Same Indiv
Different In
Create a new Named individual
Name formatoTapaBlanda
IRI emanticweb.org/user/ontologies/2026/9/untitled-ontology-5#formatoTapaBlanda
New entity options...
Aceptar
Cancelar

}



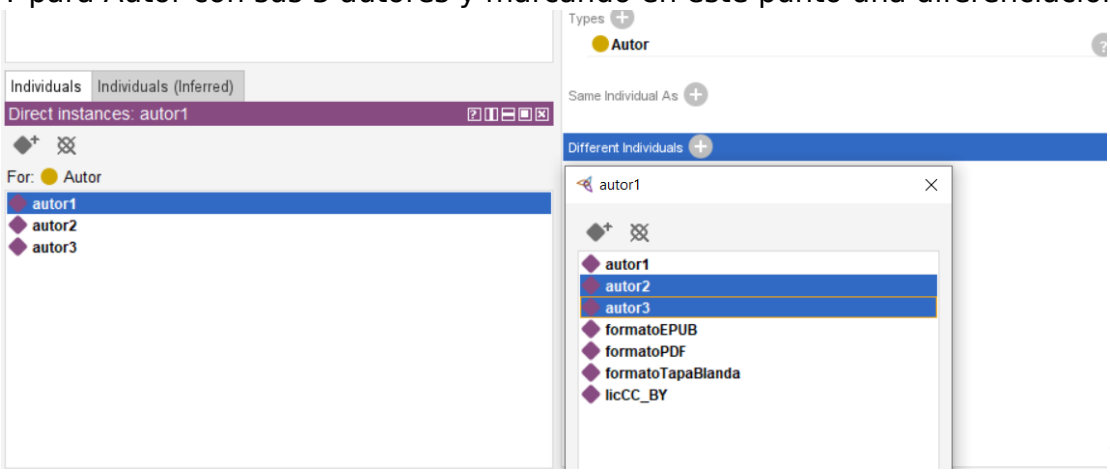
}

Lo mismo para Licencia



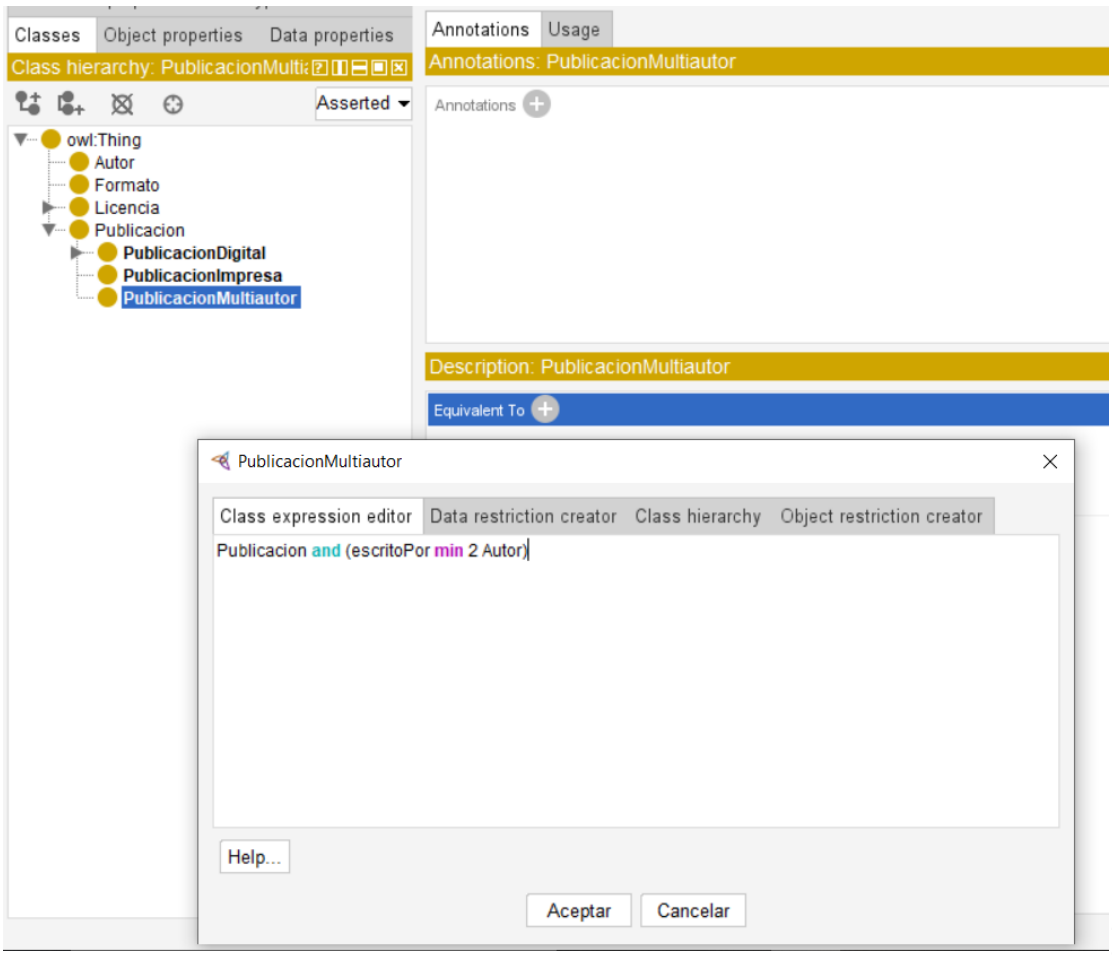
}

Y para Autor con sus 3 autores y marcando en este punto una diferenciación

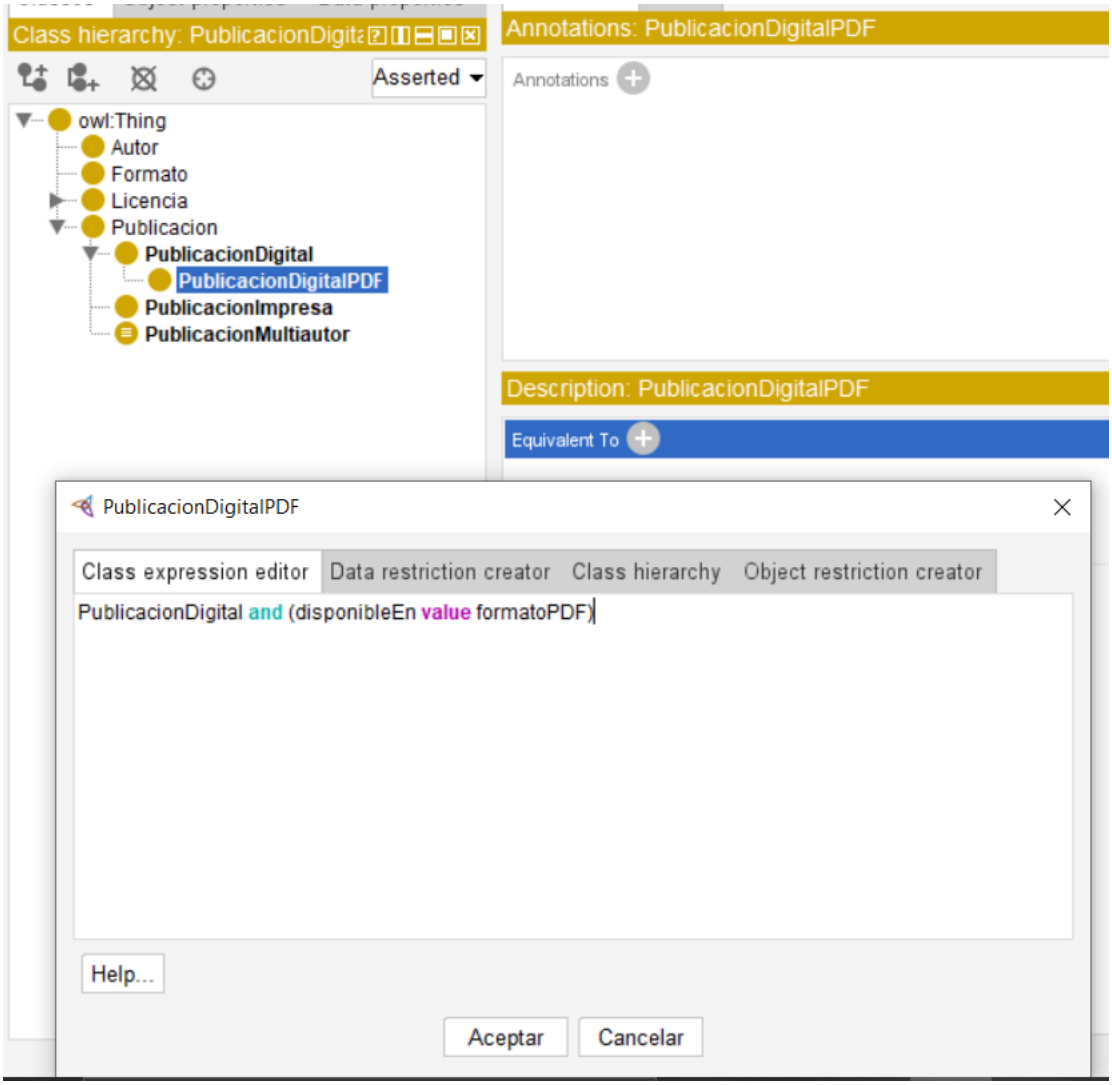


}

Pasamos a configurar las reglas lógicas de equivalencia [Equivalent to] para PublicacionMultiautor y PublicacionDigitalPDF

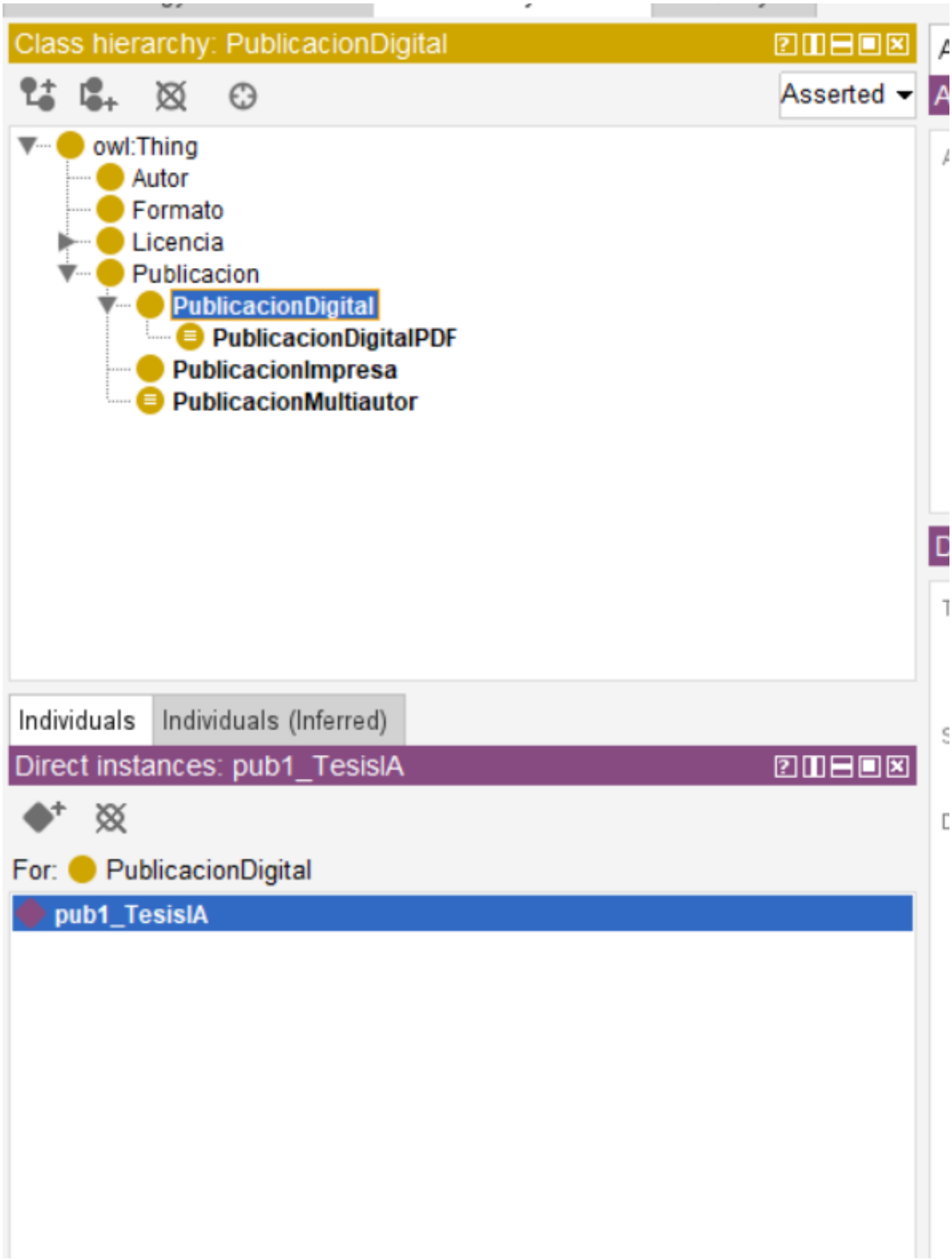

Classes
Object properties
Data properties
Class hierarchy: PublicacionMultia
Asserted
owl:Thing
Autor
Formato
Licencia
Publicacion
PublicacionDigital
PublicacionImpresa
PublicacionMultiautor
Annotations
Usage
Annotations: PublicacionMultiautor
Annotations
Description: PublicacionMultiautor
Equivalent To
PublicacionMultiautor
Class expression editor
Data restriction creator
Class hierarchy
Object restriction creator
Publicacion and (escritoPor min 2 Autor)
Help...
Aceptar
Cancelar

}



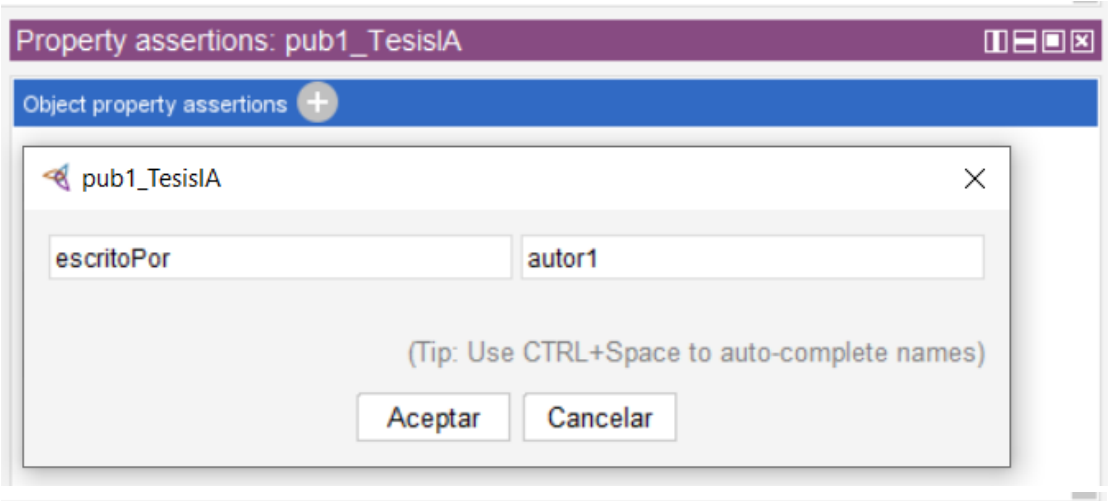
}

Continuamos con las publicaciones de prueba
pub1_TesisIA

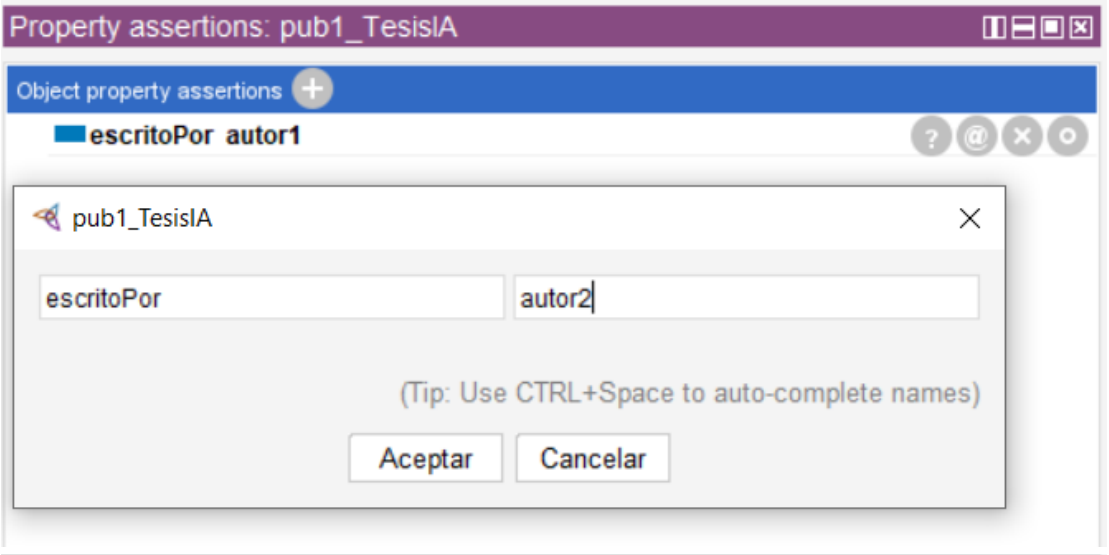


}

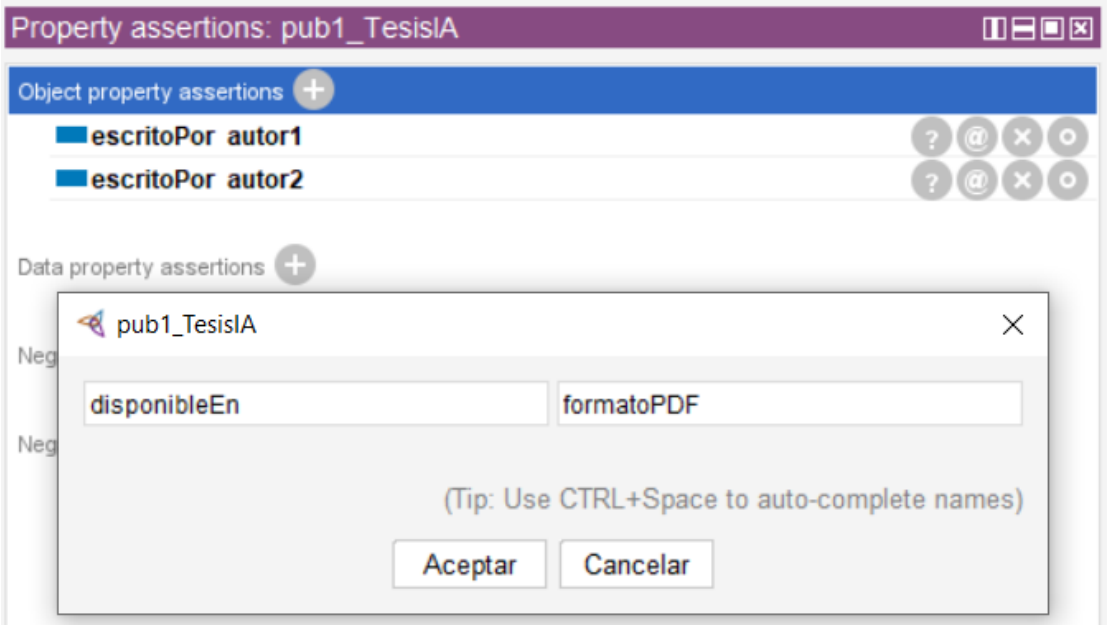
Y asignamos las relaciones [Object property assertions] y título [Data property assertions]


Property assertions: pub1_TesisIA
Object property assertions
pub1_TesisIA
escritoPor
autor1
(Tip: Use CTRL+Space to auto-complete names)
Aceptar
Cancelar

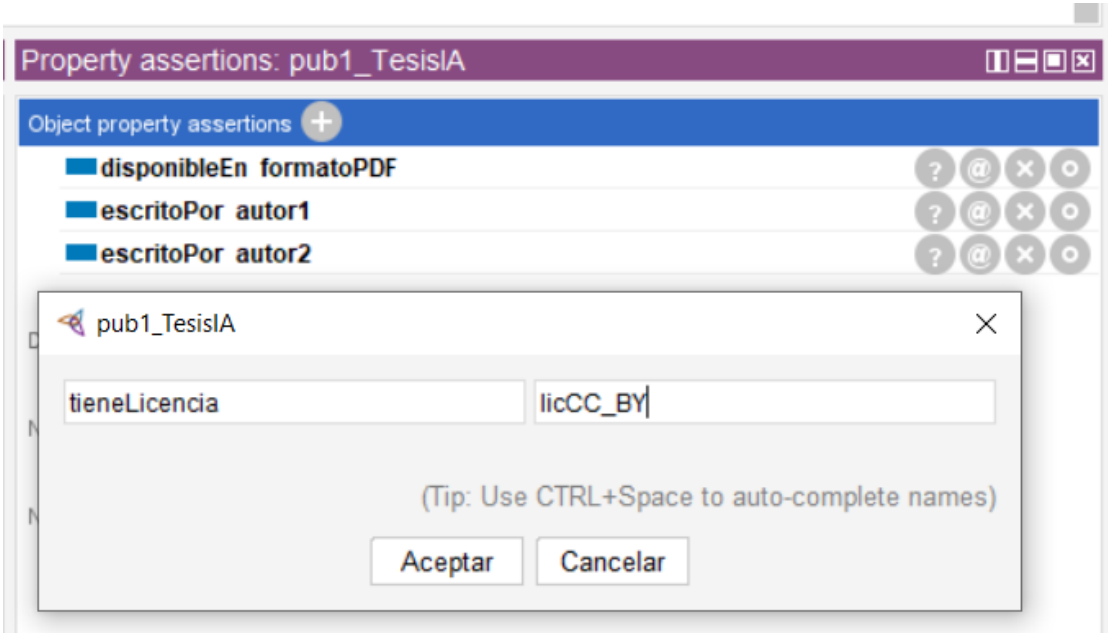
}


Property assertions: pub1_TesisIA
Object property assertions
escritoPor autor1
pub1_TesisIA
escritoPor
autor2
(Tip: Use CTRL+Space to auto-complete names)
Aceptar
Cancelar

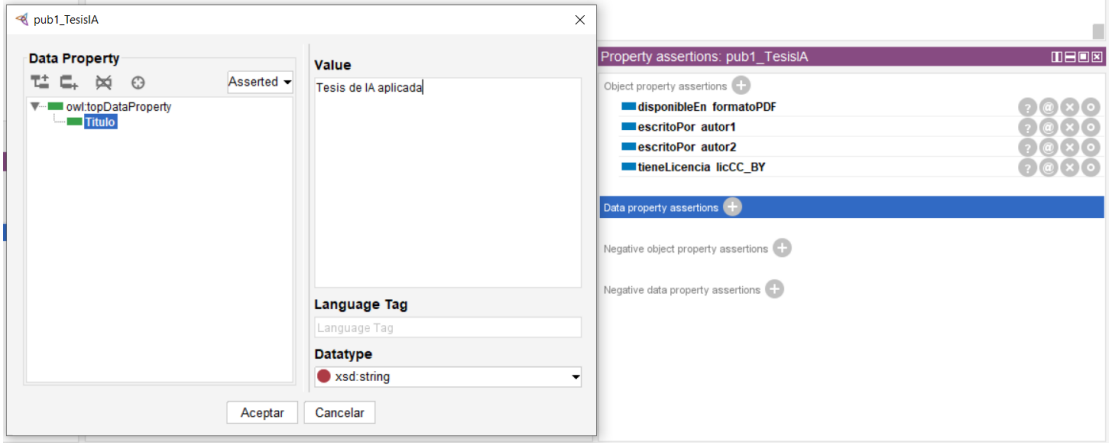
}


Property assertions: pub1_TesisIA
Object property assertions
escritoPor autor1
escritoPor autor2
Data property assertions
pub1_TesisIA
disponibleEn
formatoPDF
(Tip: Use CTRL+Space to auto-complete names)
Aceptar
Cancelar

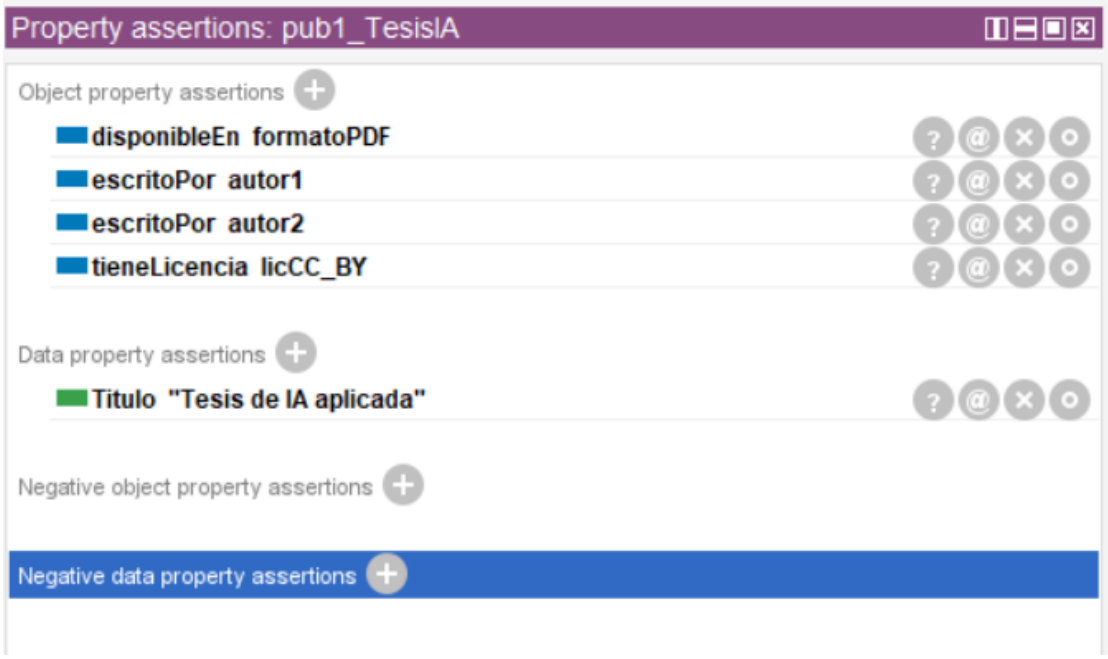
}



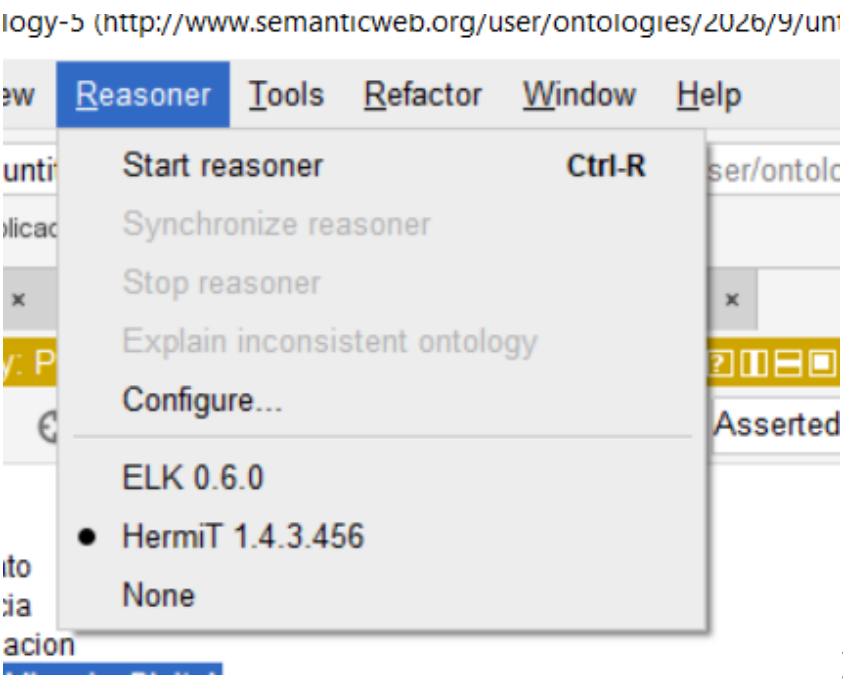
}



}

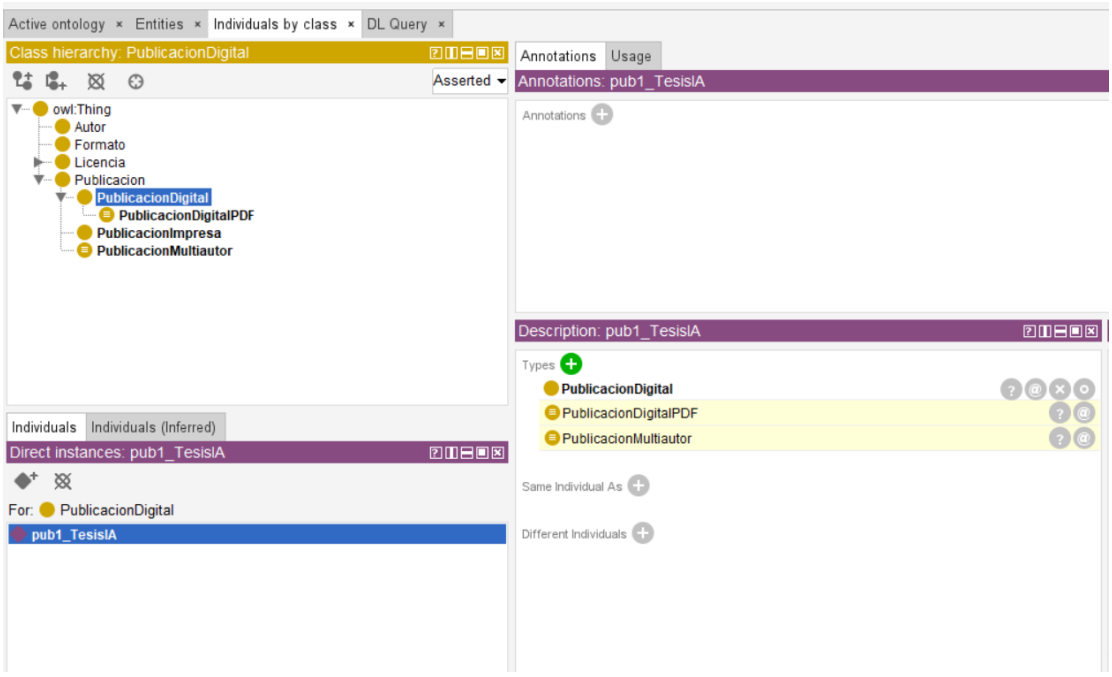


} Una vez finalizado probamos la ejecución en el modelo Hermit

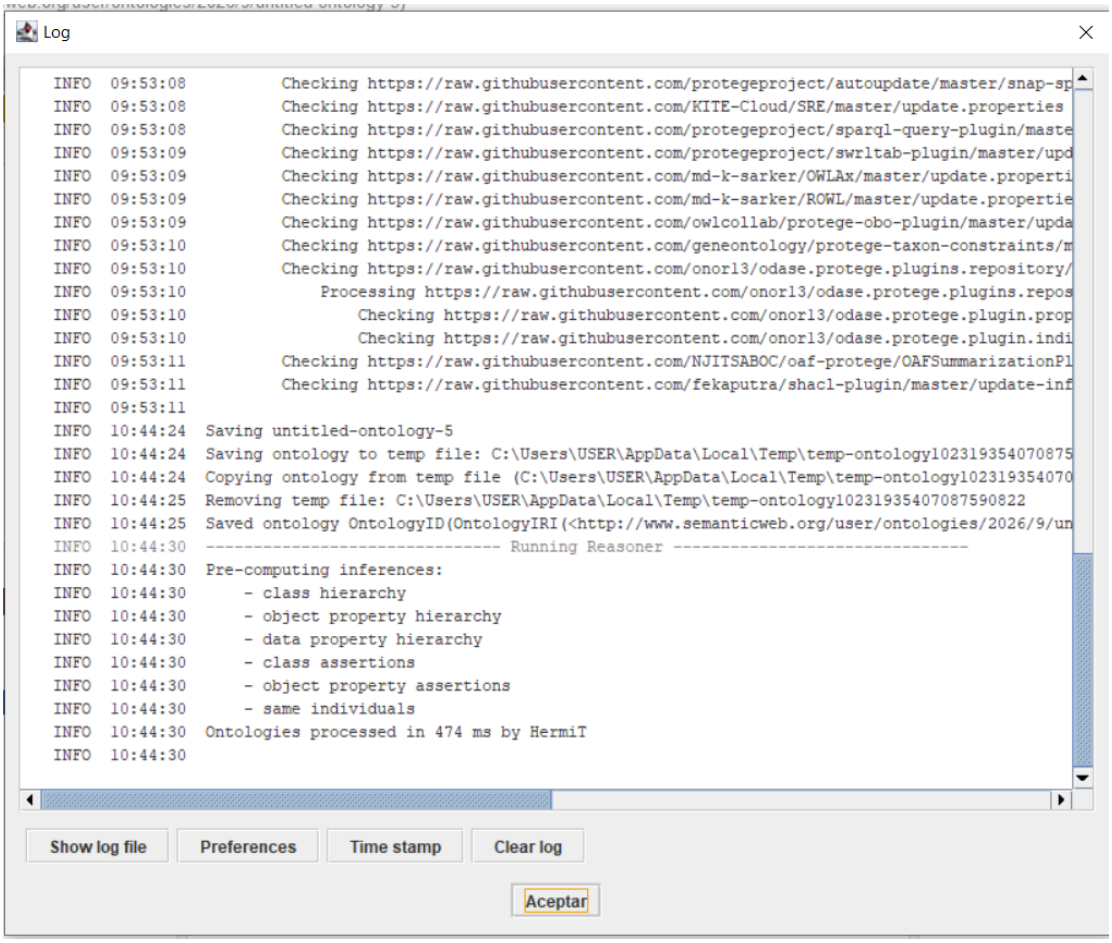

Reasoner
Tools
Refactor
Window
Help
Start reasoner
Ctrl-R
Synchronize reasoner
Stop reasoner
Explain inconsistent ontology
Configure...
ELK 0.6.0
HermiT 1.4.3.456
None
Asserted

}

Inciso c


Active ontology
Entities
Individuals by class
DL Query
Class hierarchy: PublicacionDigital
Asserted
owl:Thing
Autor
Formato
Licencia
Publicacion
PublicacionDigital
PublicacionDigitalPDF
PublicacionImpresa
PublicacionMultiautor
Annotations
Usage
Annotations: pub1_TesisIA
Annotations
Description: pub1_TesisIA
Types
PublicacionDigital
PublicacionDigitalPDF
PublicacionMultiautor
Same Individual As
Different Individuals
Individuals
Individuals (Inferred)
Direct instances: pub1_TesisIA
For: PublicacionDigital
pub1_TesisIA

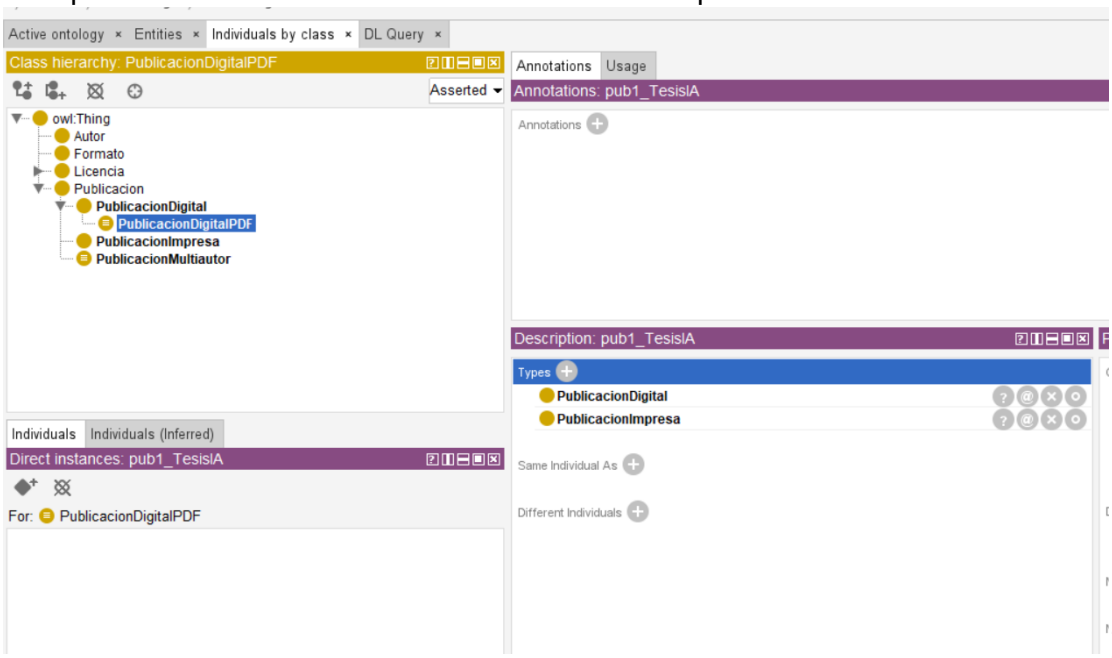
}



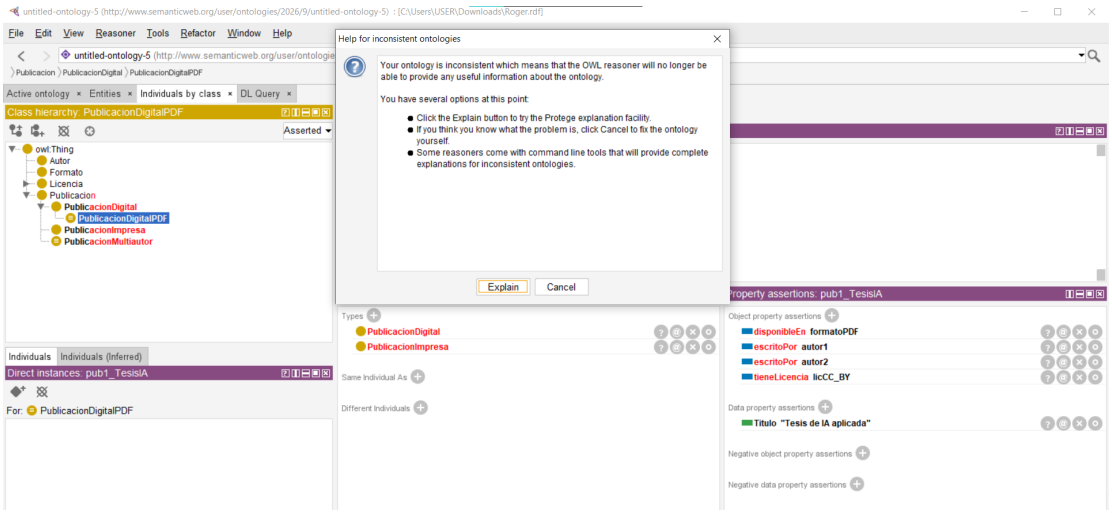
}

Inciso d

Comprobando el error añadiendo PublicacionImpresa

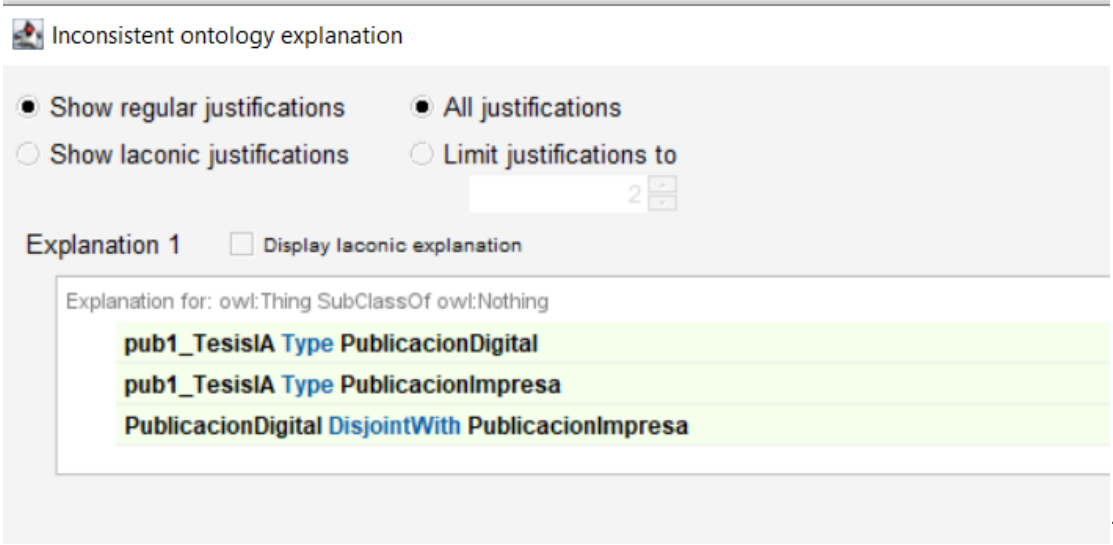


}



}

La ventana previa declara que la ontología no es consistente Mostrando justificación lógica



}

Este cuadro explica que PublicacionDigital y PublicacionImpresa son disjuntas.

4.2 Lectura de resultados (inferencias esperadas y por qué se producen)

Antes de ejecutar HermiT se esperaba:

#	Inferencia esperada	Condición que la permite
1	`pub1_TesisIA` es `PublicacionMultiautor`	Tiene 2 autores (`autor1`, `autor2`) declarados distintos (`DifferentIndividuals`) → cumple `escritoPor min 2 Autor`.
2	`pub1_TesisIA` es `PublicacionDigitalPDF`	Es `PublicacionDigital` y `disponibleEn formatoPDF` → cumple la equivalencia con `hasValue formatoPDF`.

HermiT confirma ambas: en la vista de `pub1_TesisIA` aparecen en amarillo los tipos inferidos `PublicacionDigitalPDF` y `PublicacionMultiautor`. La inferencia 1 se produce por la clase definida con cardinalidad mínima (`min 2`) combinada con individuos

distintos (sin `DifferentIndividuals`, el razonador podría fusionar `autor1` y `autor2` y no dispararía). La inferencia 2 se produce por la clase definida con `hasValue formatoPDF` sobre una publicación digital.

4.3 d) Prueba de incompatibilidad (explicación)

Al asignar a un individuo dos clases declaradas disjuntas (`PublicacionDigital` y `PublicacionImpresa`), HermiT declara la ontología inconsistente (`Help for inconsistent ontologies`) y se niega a razonar hasta corregir el modelo. La justificación lógica muestra que ambas clases son disjuntas, por lo que ningún individuo puede pertenecer a ambas a la vez.